	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Rene Adrián Davila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): Proyecto etapa 2

Integrante(s): 323001315, 426057857, 323008754, 426058665, 323067267

No. de lista o brigada: 5

Semestre: 2027-1

Fecha de entrega: 26 de Octubre

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **2**

2. Marco Teórico **3**
2.1. Entorno y lenguaje de programación 3
2.2. Fundamentos y sintaxis del lenguaje 3
2.3. Utilerías y clases de uso general 3
2.4. Clases y objetos 3
2.5. Abstracción y Encapsulamiento 4
2.6. Organización de Clases (Paquetes) 4
2.7. Herencia y Sobrescritura 4
2.8. Clases Abstractas e Interfaces 4
2.9. Polimorfismo 5

3. Desarrollo **5**
3.1. Paso 1: Descripción de Clases y Modelado (POO) 5
3.2. Paso 2: Implementación de Encapsulamiento y Control de Asignaciones 6
3.3. Paso 3: Implementación de Paquetes 6
3.4. Paso 4: Demostración de Polimorfismo 6
3.5. Paso 5: Pruebas de Funcionamiento 7

4. Resultados **9**
4.1. PruebaHospital-Etapa 2 9

5. Conclusiones **15**

1. Introducción

El desarrollo de software orientado a objetos requiere que los sistemas evolucionen para ser más seguros, modulares y escalables. Tras establecer la base funcional del simulador del sistema hospitalario en la Etapa 1, esta segunda fase se centra en reestructurar y robustecer la arquitectura del código mediante la aplicación rigurosa de los pilares de la Programación Orientada a Objetos, adaptando el sistema a una jerarquía de personal y pacientes más cercana a la realidad clínica.

- **Planteamiento del problema.** El simulador inicial carecía de protección de datos (los atributos eran de libre acceso) y no distinguía entre los diferentes niveles de especialización del personal médico ni las necesidades específicas de los pacientes. Se requiere una reestructuración que organice el código en paquetes lógicos, restrinja el acceso a la información interna mediante encapsulamiento, y modele adecuadamente los distintos tipos de roles (como médicos generales, especialistas y jefes de enfermería). Todo esto debe implementarse manteniendo intactas las reglas de negocio y restricciones de capacidad (10 pacientes por médico y 3 por enfermero) establecidas previamente.

- **Motivación.** A medida que un sistema de software crece, mantener una estructura plana con datos desprotegidos genera vulnerabilidades y código rígido. Aplicar el encapsulamiento garantiza la integridad de los datos del hospital (por ejemplo, evitar que una cédula profesional sea modificada tras su registro). Por su parte, la implementación de herencia y polimorfismo reduce drásticamente la duplicación de código y permite que el sistema sea dinámico y escalable, facilitando la interacción entre diferentes tipos de objetos desde una misma interfaz común.

- **Objetivos.** Evolucionar el simulador hospitalario integrando encapsulamiento, paquetes, herencia y polimorfismo, logrando específicamente:

  - Proteger la integridad de los objetos mediante modificadores de acceso (`private`, `protected`) y métodos de validación en constructores y *setters*.
  - Organizar las clases según su responsabilidad utilizando una arquitectura de paquetes (`hospital.sistema`, `hospital.personal`, `hospital.pacientes`, `hospital.interfaces`).
  - Diseñar e implementar clases abstractas para `Paciente`, `Medico` y `Enfermero`, así como sus respectivas subclases especializadas.
  - Definir e implementar la interfaz `AtencionMedica` para estandarizar el contrato de comportamiento del personal de salud.
  - Demostrar el funcionamiento del polimorfismo estático y dinámico en el almacenamiento y procesamiento de objetos de diferentes subclases utilizando referencias de sus clases base.

2. Marco Teórico

2.1. Entorno y lenguaje de programación

Java es un lenguaje de programación, es decir, una forma de escribir instrucciones que una computadora puede ejecutar. El programa se escribe en un archivo de texto con extensión java; el compilador (javac) lo traduce a un formato intermedio (.class) y la Máquina Virtual de Java (JVM) lo ejecuta. Gracias a la JVM, el mismo programa funciona en distintos sistemas operativos. [1]

2.2. Fundamentos y sintaxis del lenguaje

La sintaxis son las reglas de escritura del lenguaje: en Java cada instrucción termina con punto y coma, los bloques se encierran entre llaves y se distingue entre mayúsculas y minúsculas. Cada dato tiene un tipo; por ejemplo, int guarda números enteros y String guarda texto. Para tomar decisiones se usan if-else y switch; para repetir instrucciones, for y do-while. La clase Scanner lee lo que el usuario escribe en el teclado.[1],[3]

2.3. Utilerías y clases de uso general

Un arreglo es una lista de tamaño fijo cuyos elementos son del mismo tipo; cada posición se identifica con un número que empieza en 0. Por ejemplo, un arreglo de 10 posiciones sirve para guardar hasta 10 pacientes. Un TreeSet es una colección que no permite elementos repetidos y ordena alfabéticamente los textos que se le agregan. Un texto es un objeto de la clase String y se compara con el método equals. [3]

2.4. Clases y objetos

Una clase es un molde que describe cómo es un tipo de elemento: qué datos tiene y qué puede hacer. Un objeto es un elemento concreto creado a partir de ese molde con la palabra new. Por ejemplo, la clase Paciente es la ficha en blanco, y cada paciente registrado es una ficha ya llena.[2]

Los atributos son los datos de un objeto (por ejemplo, el nombre de un paciente). Los métodos son las acciones que puede realizar (por ejemplo, darConsulta). Un objeto le pide una acción a otro escribiendo el nombre del objeto, un punto y el método. Dentro de una clase, this se refiere al propio objeto. [2]

Un constructor es un método con el mismo nombre que la clase, que se ejecuta al crear un objeto y le da sus valores iniciales. [2]

La sobrecarga consiste en tener varios métodos con el mismo nombre pero con datos de entrada distintos; Java elige cuál usar según lo que recibe. En este proyecto, asignar Paciente existe en una versión para médicos y otra para enfermeros. [1]

2.5. Abstracción y Encapsulamiento

La abstracción es la propiedad que considera los aspectos más significativos de un problema y expresa una solución en esos términos, ignorando los detalles internos. El encapsulamiento es el proceso de agrupamiento de datos y operaciones bajo una misma unidad, ocultando los datos del mundo exterior. Para esto se utilizan modificadores de acceso:[1]

- **private:** Nivel más fuerte de protección; totalmente invisible para otras clases.[1]
- **protected:** Permite que los atributos sean visibles para la misma clase, clases derivadas y clases del mismo paquete.[1]
- **public:** El nivel más débil, hace que los elementos sean visibles para todas las clases.[1]

El acceso a los atributos encapsulados se realiza mediante métodos consultores (*getters*) y modificadores (*setters*), permitiendo restringir y validar la modificación de los datos.[1]

2.6. Organización de Clases (Paquetes)

Un paquete (*package*) es una agrupación de clases que se utiliza para organizar elementos relacionados lógicamente, evitar conflictos de nombres y ayudar en el control de accesibilidad. Las clases de un sistema se almacenan en jerarquías de directorios que coinciden con el nombre compuesto del paquete. Para hacer que las clases en un paquete particular estén disponibles, es necesario importar ese paquete usando `import` y su nombre calificado (*Fully Qualified Name*).[1]

2.7. Herencia y Sobrescritura

La herencia es el proceso que implica la creación de clases a partir de clases ya existentes (clases base o superclases), permitiendo promover la reutilización de código. Una clase derivada hereda todas las propiedades y métodos visibles mediante la palabra reservada `extends`, estableciendo una relación *IS-A* (es un).[1]

La sobrescritura (*overriding*) permite redefinir el comportamiento de un método específico heredado en una subclase, manteniendo el mismo nombre, tipo y número de parámetros. Además, los constructores de las clases derivadas pueden invocar explícitamente a los de su clase base utilizando `super`.[1]

2.8. Clases Abstractas e Interfaces

Una clase abstracta es aquella de la cual no se pueden crear objetos (instanciar), debido a que define la existencia de métodos, pero no su implementación completa, sirviendo como modelo para las clases derivadas.[1]

Una interfaz es una clase abstracta pura donde todos los métodos son abstractos y públicos. Actúa como un contrato sobre qué puede hacer la clase sin dictar cómo lo hará. Las clases implementan estas características mediante la palabra reservada `implements`, permitiendo la implementación múltiple.[1]

2.9. Polimorfismo

El polimorfismo es la propiedad por la cual es posible enviar mensajes sintácticamente iguales a objetos de tipos distintos. En Java, a una referencia se le puede asignar cualquier objeto que sea del mismo tipo con el que fue declarada o de algún subtipo (polimorfismo dinámico). Al invocar un método, el programa ejecuta la versión correspondiente al tipo real del objeto instanciado, facilitando el procesamiento genérico de jerarquías de clases.[1]

3. Desarrollo

3.1. Paso 1: Descripción de Clases y Modelado (POO)

Se rediseñó la arquitectura del sistema incorporando clases abstractas, herencia e interfaces:

- **Paciente (Clase Abstracta):** Almacena los datos básicos encapsulados (nombre, apellidos, especialidad y estado). Define el método abstracto `solicitarConsulta()`, el cual es implementado por sus subclases `PacienteGeneral` y `PacienteEspecialidad`, agregando atributos propios como `motivoConsulta` y `diagnosticoPrevio` respectivamente.
- **Medico y Enfermero (Clases Abstractas):** Representan al personal de salud y almacenan a sus pacientes asignados mediante arreglos dinámicos (`ArrayList`). Implementan la interfaz `AtencionMedica` y definen métodos abstractos como `puedeAtender()`, delegando la implementación específica a sus subclases (`MedicoGeneral`, `MedicoEspecialista`, `EnfermeroGeneral`, `EnfermeroEspecialista` y `JefeEnfermeria`).
- **AtencionMedica (Interfaz):** Establece el contrato de comportamiento obligatorio para todo el personal de salud, estandarizando los métodos `darTratamiento()` y `verListaPacientes()`.
- **Sistema:** Clase controladora encargada de coordinar los registros globales utilizando polimorfismo dinámico mediante colecciones genéricas.
- **PruebaHospital:** Contiene la interfaz de consola interactiva.

3.2. Paso 2: Implementación de Encapsulamiento y Control de Asignaciones

Para asegurar la integridad de la información y gestionar las capacidades, se implementaron las siguientes restricciones de acceso:

- **Protección de Datos:** Todos los atributos de las clases se configuraron con visibilidad `private`. El acceso y modificación se realiza exclusivamente a través de métodos *getters* y *setters*, los cuales integran validaciones para rechazar valores nulos o cadenas vacías.
- **Atributos Inmutables:** Como decisión de diseño, atributos de identidad como `cedula` y `nombre` se inicializan en el constructor y carecen de métodos `set`, garantizando que no sean alterados durante el ciclo de vida del objeto.
- **Manejo de Capacidad y Duplicados:** Se sustituyeron los arreglos estáticos por colecciones dinámicas (`ArrayList`), manteniendo las validaciones lógicas para respetar los límites de 10 pacientes por médico y 3 por enfermero, y verificando la inexistencia de duplicados antes de cada asignación.

3.3. Paso 3: Implementación de Paquetes

El código fuente se organizó en una estructura jerárquica de paquetes para separar las responsabilidades lógicas y controlar la accesibilidad:

- `hospital.sistema`: Aloja la clase controladora.
- `hospital.personal`: Agrupa las jerarquías del personal de salud.
- `hospital.pacientes`: Administra las clases de pacientes.
- `hospital.interfaces`: Contiene los contratos de comportamiento.

La integración de estos componentes en la clase principal se realizó mediante declaraciones `import` y el uso de Nombres Calificados (*Fully Qualified Names*).

3.4. Paso 4: Demostración de Polimorfismo

El simulador aprovecha las capacidades polimórficas del lenguaje para procesar objetos de distintas clases de manera uniforme:

- **Polimorfismo Dinámico en Colecciones:** La clase `Sistema` almacena los registros en listas genéricas referenciadas por sus clases base (`ArrayList<Paciente>`). Al recorrer estas colecciones y ejecutar un método (ej. `solicitarConsulta()`), el sistema determina en tiempo de ejecución a qué subclase pertenece el objeto (`PacienteGeneral` o `PacienteEspecialidad`) y ejecuta su versión específica.
- **Polimorfismo mediante Interfaces:** Se agrupa a todo el personal registrado (`Medico` y `Enfermero`) en una lista de tipo `AtencionMedica`. Al invocar `verListaPacientes()` sobre esta lista, cada objeto responde según su propia implementación, demostrando la flexibilidad de las interfaces.

3.5. Paso 5: Pruebas de Funcionamiento

Se probó la ejecución del sistema validando las nuevas operaciones incorporadas:

1. **Validación de Herencia y Asignación Especializada:** Se registró un `MedicoEspecialista` (Cardiología) y se intentó asignarle un `PacienteGeneral`. El sistema rechazó la asignación gracias a la validación polimórfica del método `puedeAtender()`. Posteriormente, se le asignó un `PacienteEspecialidad` (Cardiología), confirmando el registro exitoso.

2. **Ejecución Polimórfica:** Se registraron pacientes generales y de especialidad, y se invocó la opción de solicitar consulta global. El sistema imprimió correctamente el motivo de consulta para los pacientes generales y el diagnóstico previo para los pacientes de especialidad, demostrando la resolución dinámica de métodos.

3. **Supervisión Jerárquica:** Se registró un `JefeEnfermeria` y se le asignaron enfermeros generales a su equipo. Al invocar la función de supervisión, el sistema imprimió correctamente el estado de carga de pacientes de su equipo a cargo.

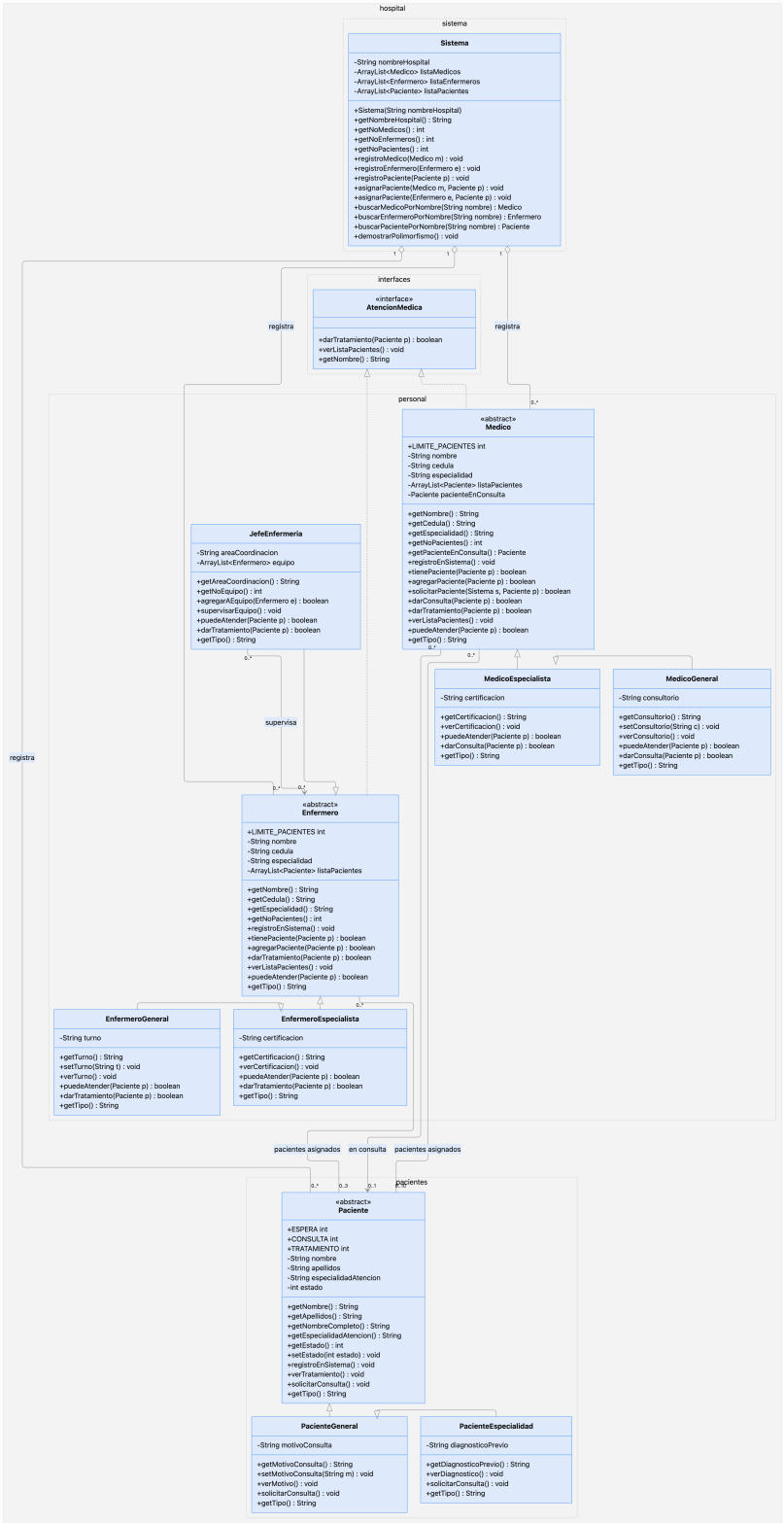


Figura 1: Diagrama de clases etapa 2

4. Resultados

4.1. PruebaHospital-Etapa 2

El programa es un simulador interactivo en consola que gestiona las operaciones de un sistema hospitalario mediante un menú de opciones ampliado. En esta segunda etapa, el sistema incorpora jerarquías de herencia para clasificar de manera precisa al personal de salud (médicos generales, médicos especialistas, enfermeros generales, enfermeros especialistas y jefes de enfermería) y a los pacientes (generales y de especialidad). A través del polimorfismo dinámico y la implementación de la interfaz `AtencionMedica`, el simulador valida automáticamente que las asignaciones correspondan estrictamente al tipo de paciente y la rama médica del profesional, respetando los límites de capacidad establecidos (10 pacientes por médico y 3 por enfermero). Además de rastrear el estado de los pacientes y generar listas ordenadas descendentemente, el sistema demuestra la resolución dinámica de métodos al ejecutar comportamientos específicos para cada rol, tales como la emisión de diagnósticos detallados, o la supervisión de la carga de pacientes por parte del jefe de enfermería.

```
PS C:\Users\acer\Downloads\py2_etapa2\poo\g7\e0> cd py2
PS C:\Users\acer\Downloads\py2_etapa2\poo\g7\e0\py2> javac -cp src test\PruebaHospital.java
PS C:\Users\acer\Downloads\py2_etapa2\poo\g7\e0\py2> java -cp "src;test" PruebaHospital

===== MENU Hospital General POO =====
1. Registrar medico (general o especialista)
2. Registrar enfermero (general, especialista o jefe)
3. Registrar paciente (general o de especialidad)
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
12. Jefe de enfermeria: Agregar enfermero a su equipo
13. Jefe de enfermeria: Supervisar equipo
14. Demostracion de polimorfismo
0. Salir
Elige una opcion: 1
Tipo de medico (1=General, 2=Especialista): 2
Nombre del medico: Simi
Cedula: 123
Especialidad: Cardiologia
Certificacion: Consejomex
Médico Simi registrado (Médico especialista). Especialidad: Cardiologia
```

Figura 2: Resultados

```
===== MENU Hospital General POO =====
1. Registrar medico (general o especialista)
2. Registrar enfermero (general, especialista o jefe)
3. Registrar paciente (general o de especialidad)
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
12. Jefe de enfermeria: Agregar enfermero a su equipo
13. Jefe de enfermeria: Supervisar equipo
14. Demostracion de polimorfismo
0. Salir
Elige una opcion: 3
Tipo de paciente (1=General, 2=Especialidad): 2
Nombre del paciente: Juan
Apellidos del paciente: Perez
Especialidad que necesita: Cardiologia
Diagnostico previo: Arritmia
Paciente registrado: Perez Juan (Paciente de especialidad)
```

Figura 3: Resultados

```
===== MENU Hospital General POO =====
1. Registrar medico (general o especialista)
2. Registrar enfermero (general, especialista o jefe)
3. Registrar paciente (general o de especialidad)
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
12. Jefe de enfermeria: Agregar enfermero a su equipo
13. Jefe de enfermeria: Supervisar equipo
14. Demostracion de polimorfismo
0. Salir
Elige una opcion: 3
Tipo de paciente (1=General, 2=Especialidad): 1
Nombre del paciente: Ana
Apellidos del paciente: Lopez
Motivo de consulta: Fiebre
Paciente registrado: Lopez Ana (Paciente general)
```

Figura 4: Resultados

```
===== MENU Hospital General POO =====
1. Registrar medico (general o especialista)
2. Registrar enfermero (general, especialista o jefe)
3. Registrar paciente (general o de especialidad)
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
12. Jefe de enfermeria: Agregar enfermero a su equipo
13. Jefe de enfermeria: Supervisar equipo
14. Demostracion de polimorfismo
0. Salir
Elige una opcion: 4
Nombre del medico: Simi
Nombre del paciente a asignar: Ana
Error: La especialidad no coincide (Médico especialista de Cardiologia; el paciente requiere General).
```

Figura 5: Resultados

```
===== MENU Hospital General POO =====
1. Registrar medico (general o especialista)
2. Registrar enfermero (general, especialista o jefe)
3. Registrar paciente (general o de especialidad)
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
12. Jefe de enfermeria: Agregar enfermero a su equipo
13. Jefe de enfermeria: Supervisar equipo
14. Demostracion de polimorfismo
0. Salir
Elige una opcion: 4
Nombre del medico: Simi
Nombre del paciente a asignar: Juan
Paciente asignado al médico Simi
```

Figura 6: Resultados

```
===== MENU Hospital General POO =====
1. Registrar medico (general o especialista)
2. Registrar enfermero (general, especialista o jefe)
3. Registrar paciente (general o de especialidad)
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
12. Jefe de enfermeria: Agregar enfermero a su equipo
13. Jefe de enfermeria: Supervisar equipo
14. Demostracion de polimorfismo
0. Salir
Elige una opcion: 14

--- Pacientes (referencia de tipo Paciente) ---
[Paciente de especialidad]
El paciente Juan solicita consulta de la especialidad Cardiologia. Diagnóstico previo: Arritmia
[Paciente general]
El paciente Ana solicita consulta de medicina general. Motivo: Fiebre

--- Médicos (referencia de tipo Medico) ---
Médico especialista: Simi - pacientes: 1/10

--- Enfermeros (referencia de tipo Enfermero) ---

--- Todo el personal (referencia de tipo AtencionMedica) ---

Lista de pacientes de Simi (Orden Descendente):
Perez Juan - Cardiologia
```

Figura 7: Resultados

```
===== MENU Hospital General POO =====
1. Registrar medico (general o especialista)
2. Registrar enfermero (general, especialista o jefe)
3. Registrar paciente (general o de especialidad)
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
12. Jefe de enfermeria: Agregar enfermero a su equipo
13. Jefe de enfermeria: Supervisar equipo
14. Demostracion de polimorfismo
0. Salir
Elige una opcion: 2
Tipo de enfermero (1=General, 2=Especialista, 3=Jefe): 3
Nombre del enfermero: Joy
Cedula: 456
Area que coordina: Urgencias
Enfermero Joy registrado (Jefe de enfermería). Especialidad: General
```

Figura 8: Resultados

```
===== MENU Hospital General POO =====
1. Registrar medico (general o especialista)
2. Registrar enfermero (general, especialista o jefe)
3. Registrar paciente (general o de especialidad)
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
12. Jefe de enfermeria: Agregar enfermero a su equipo
13. Jefe de enfermeria: Supervisar equipo
14. Demostracion de polimorfismo
0. Salir
Elige una opcion: 2
Tipo de enfermero (1=General, 2=Especialista, 3=Jefe): 1
Nombre del enfermero: Luis
Cedula: 789
Turno: Matutino
Enfermero Luis registrado (Enfermero general). Especialidad: General
```

Figura 9: Resultados

```
===== MENU Hospital General POO =====
1. Registrar medico (general o especialista)
2. Registrar enfermero (general, especialista o jefe)
3. Registrar paciente (general o de especialidad)
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
12. Jefe de enfermeria: Agregar enfermero a su equipo
13. Jefe de enfermeria: Supervisar equipo
14. Demostracion de polimorfismo
0. Salir
Elige una opcion: 12
Nombre del jefe de enfermeria: Joy
Nombre del enfermero a agregar: Luis
Luis fue agregado al equipo de Joy.
```

Figura 10: Resultados

```
===== MENU Hospital General POO =====
1. Registrar medico (general o especialista)
2. Registrar enfermero (general, especialista o jefe)
3. Registrar paciente (general o de especialidad)
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Ver lista de pacientes de un medico
7. Ver lista de pacientes de un enfermero
8. Paciente: Ver estado de tratamiento
9. Paciente: Solicitar consulta
10. Medico: Solicitar paciente y dar consulta
11. Medico/Enfermero: Dar tratamiento
12. Jefe de enfermeria: Agregar enfermero a su equipo
13. Jefe de enfermeria: Supervisar equipo
14. Demostracion de polimorfismo
0. Salir
Elige una opcion: 13
Nombre del jefe de enfermeria: Joy

Supervisión del equipo de Joy (área: Urgencias):
  1. Luis (Enfermero general) - pacientes: 0/3
```

Figura 11: Resultados

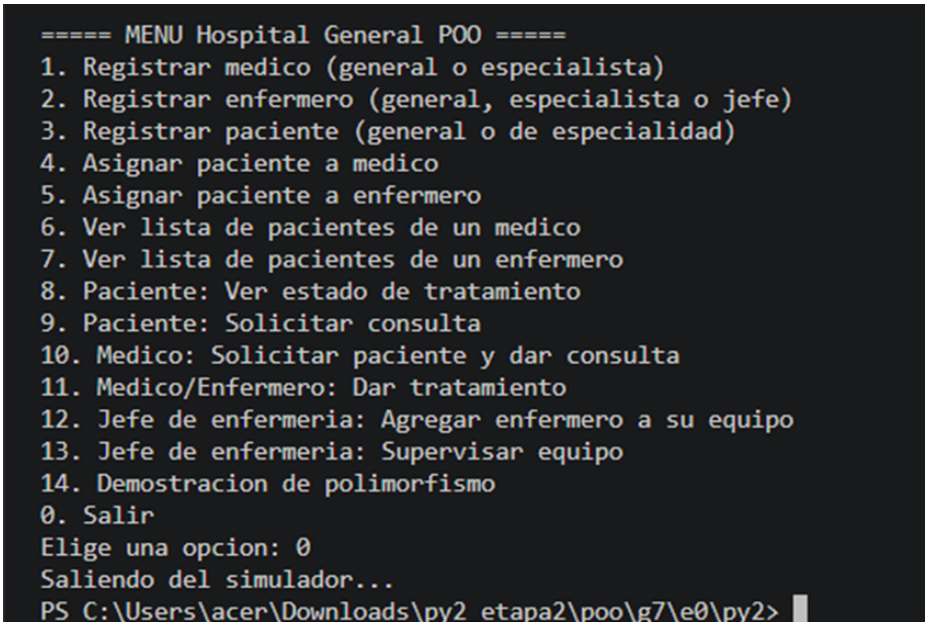


Figura 12: Resultados

5. Conclusiones

El desarrollo de la segunda etapa del simulador demostró que la reestructuración del software mediante los pilares avanzados de la Programación Orientada a Objetos es fundamental para garantizar su escalabilidad y seguridad. El análisis de esta implementación destaca lo siguiente:

- **Integridad y Modularidad:** La aplicación estricta del encapsulamiento resolvió la vulnerabilidad de los datos protegiendo la información sensible mediante modificadores de acceso y validaciones. Simultáneamente, la distribución en paquetes delimitó claramente las responsabilidades del sistema.
- **Reutilización y Reglas de Negocio:** El uso de herencia y clases abstractas (`Paciente`, `Medico`, `Enfermero`) erradicó la duplicación de código. Esto permitió modelar con precisión roles específicos y cumplir con sus restricciones reales, como la función de supervisión exclusiva del jefe de enfermería.
- **Procesamiento Dinámico:** La interfaz `AtencionMedica` y el polimorfismo dinámico simplificaron drásticamente la clase controladora. Agrupar distintas entidades bajo un contrato común permitió resolver las acciones correctas en tiempo de ejecución dependiendo del tipo real del objeto instanciado.

En conjunto, este diseño no solo satisface los requerimientos actuales del problema, sino que consolida una arquitectura robusta, capaz de asimilar nuevas funcionalidades sin alterar su base operativa.

Referencias

[1] M. A. Nakayama C. y J. A. Solano Gálvez, *Manual de prácticas del Laboratorio de Modelos de Programación Orientada a Objetos (MADO-21)*, versión 02. Ciudad de México: Facultad de Ingeniería, UNAM, 2018.

[2] L. Joyanes Aguilar, *Fundamentos de programación. Algoritmos, estructuras de datos y objetos*, 4.ª ed. México: McGraw-Hill, 2008.

[3] J. Dean y R. Dean, *Introducción a la programación con Java*, 1.ª ed. México: McGraw-Hill, 2009.